

Persistencia de Datos y Gestión de Estado

Back-End Development

Carlos Rojas-Sánchez

Universidad del Mar

Contenido

1. Integración con bases de datos relacionales

2. Pool de conexiones con Node.js y MariaDB

3. Mapeo objeto-relacional (ORM)

4. Migraciones de base de datos

5. Caché y almacenes NoSQL

Integración con bases de datos relacionales

Conexiones y la sobrecarga de red

El costo de crear conexiones

Cada nueva conexión TCP a la base de datos implica un *handshake* de red, autenticación de usuario, asignación de memoria en el servidor y configuración de sesión.

- **Conexión directa por petición**: Crear y cerrar una conexión en cada *request* causa latencia severa y agotamiento de sockets en el servidor.
- **Límite de procesos**: Las bases de datos SQL (ej. PostgreSQL, MySQL) tienen un número máximo de conexiones concurrentes permitidas.
- **Estrategia**: Mantenimiento de conexiones persistentes reusables.

Pools de conexiones (Connection Pooling)

Concepto

Un **Pool** es un caché administrado de conexiones a la base de datos abiertas y listas para ser reutilizadas por la aplicación.

- **Min / Max Pool Size**: Define el número mínimo de conexiones inactivas a mantener y el límite máximo permitido.
- **Tiempo de espera (Idle Timeout)**: Cierra conexiones inactivas para liberar recursos.
- **Cola de solicitudes (Queueing)**: Si el pool está agotado, las peticiones entrantes esperan hasta que una conexión sea liberada.

Sentencias preparadas (Prepared Statements)

Optimizando la ejecución y la seguridad

Permiten a la base de datos compilar y planificar la ejecución de una consulta SQL de forma anticipada.

- **Prevención de Inyección SQL**: Los parámetros introducidos por el usuario se tratan estrictamente como datos, nunca como código ejecutable.
- **Rendimiento**: La base de datos analiza (*parse*) y optimiza la consulta una sola vez; luego la ejecuta repetidamente cambiando solo las variables.
- **Buena práctica**: NUNCA concatenar cadenas de texto en consultas SQL.

Transacciones ACID en aplicaciones

Propiedades ACID

Garantizan la fiabilidad de las operaciones sobre la base de datos.

- **Atemicidad (Atomicity)**: Todo o nada. Si falla un paso, se ejecuta un `ROLLBACK`. Si todos tienen éxito, un `COMMIT`.
- **Consistencia (Consistency)**: La base de datos pasa de un estado válido a otro, respetando restricciones de integridad.
- **Aislamiento (Isolation)**: Las transacciones concurrentes no interfieren entre sí.
- **Durabilidad (Durability)**: Los cambios confirmados persisten ante fallos de energía o caídas del sistema.

Niveles de aislamiento de transacciones

Rigen el comportamiento y la visibilidad de los datos entre transacciones concurrentes:

- **Read Uncommitted**: Permite lecturas sucias (*dirty reads*). Rápido pero peligroso.
- **Read Committed** (Predeterminado en PostgreSQL): Evita lecturas sucias, pero permite lecturas no repetibles.
- **Repeatable Read**: Garantiza que los datos leídos no cambien durante el transcurso de la transacción.
- **Serializable**: El nivel más estricto. Simula una ejecución secuencial estricta de las transacciones (previene lecturas fantasma).

Práctica: Inicialización de proyectos con PostgreSQL

Entorno Node.js (**pg**)

```
# Crear directorio y entrar
mkdir db-relacional && cd db-relacional

# Inicializar proyecto Node.js
npm init -y
npm pkg set type="module"

# Instalar driver de PostgreSQL
npm install pg dotenv
```

Entorno Python (**psycopg2**)

```
# Crear directorio y entorno
mkdir db-relacional && cd db-relacional
python -m venv venv
source venv/bin/activate

# Instalar driver y utilidades
pip install psycopg2-binary \
            python-dotenv
```

Práctica: Pool de conexiones y Sentencias preparadas en Node.js

```
import pg from 'pg';
import dotenv from 'dotenv';
dotenv.config();

const { Pool } = pg;
const pool = new Pool({
  connectionString: process.env.DATABASE_URL,
  max: 10,
  idleTimeoutMillis: 30000
});

export async function obtenerUsuarioPorId(id) {
  // Sentencia preparada con parámetros pasados por arreglo ($1)
  const text = 'SELECT id, nombre, email FROM usuarios WHERE id = $1';
  const values = [id];
  const result = await pool.query(text, values);
  return result.rows[0];
}
```

Práctica: Transacciones ACID en Node.js

```
export async function transferirFondos(origenId, destinoId, monto) {
  const client = await pool.connect();
  try {
    await client.query('BEGIN'); // Inicio de transacción
    await client.query(
      'UPDATE cuentas SET saldo = saldo - $1 WHERE id = $2', [monto, origenId]
    );
    await client.query(
      'UPDATE cuentas SET saldo = saldo + $1 WHERE id = $2', [monto, destinoId]
    );
    await client.query('COMMIT'); // Confirmación
  } catch (err) {
    await client.query('ROLLBACK'); // Reversión ante fallo
    throw err;
  } finally {
    client.release(); // Devolver conexión al pool
  }
}
```

Práctica: Pool y Sentencias preparadas en Python

```
import os
from psycopg2 import pool
from dotenv import load_dotenv

load_dotenv()

# Pool de conexiones ThreadedConnectionPool
db_pool = pool.ThreadedConnectionPool(
    minconn=1, maxconn=10, dsn=os.getenv("DATABASE_URL")
)

def obtener_usuario(user_id: int):
    conn = db_pool.getconn()
    try:
        with conn.cursor() as cursor:
            # Sentencia preparada usando sintaxis de marcadores (%s)
            sql = "SELECT id, nombre FROM usuarios WHERE id = %s;"
            cursor.execute(sql, (user_id,))
            return cursor.fetchone()
    finally:
        db_pool.putconn(conn)
```

Práctica: Transacciones ACID en Python

```
def transferir_fondos(origen_id: int, destino_id: int, monto: float):
    conn = db_pool.getconn()
    try:
        with conn.cursor() as cursor:
            cursor.execute(
                "UPDATE cuentas SET saldo = saldo - %s WHERE id = %s;",
                (monto, origen_id)
            )
            cursor.execute(
                "UPDATE cuentas SET saldo = saldo + %s WHERE id = %s;",
                (monto, destino_id)
            )
        conn.commit() # Confirma todos los comandos en la transacción
    except Exception as e:
        conn.rollback() # Revierte todos los cambios si ocurre un error
        raise e
    finally:
        db_pool.putconn(conn)
```

Pool de conexiones con Node.js y MariaDB

¿Qué es un pool de conexiones?

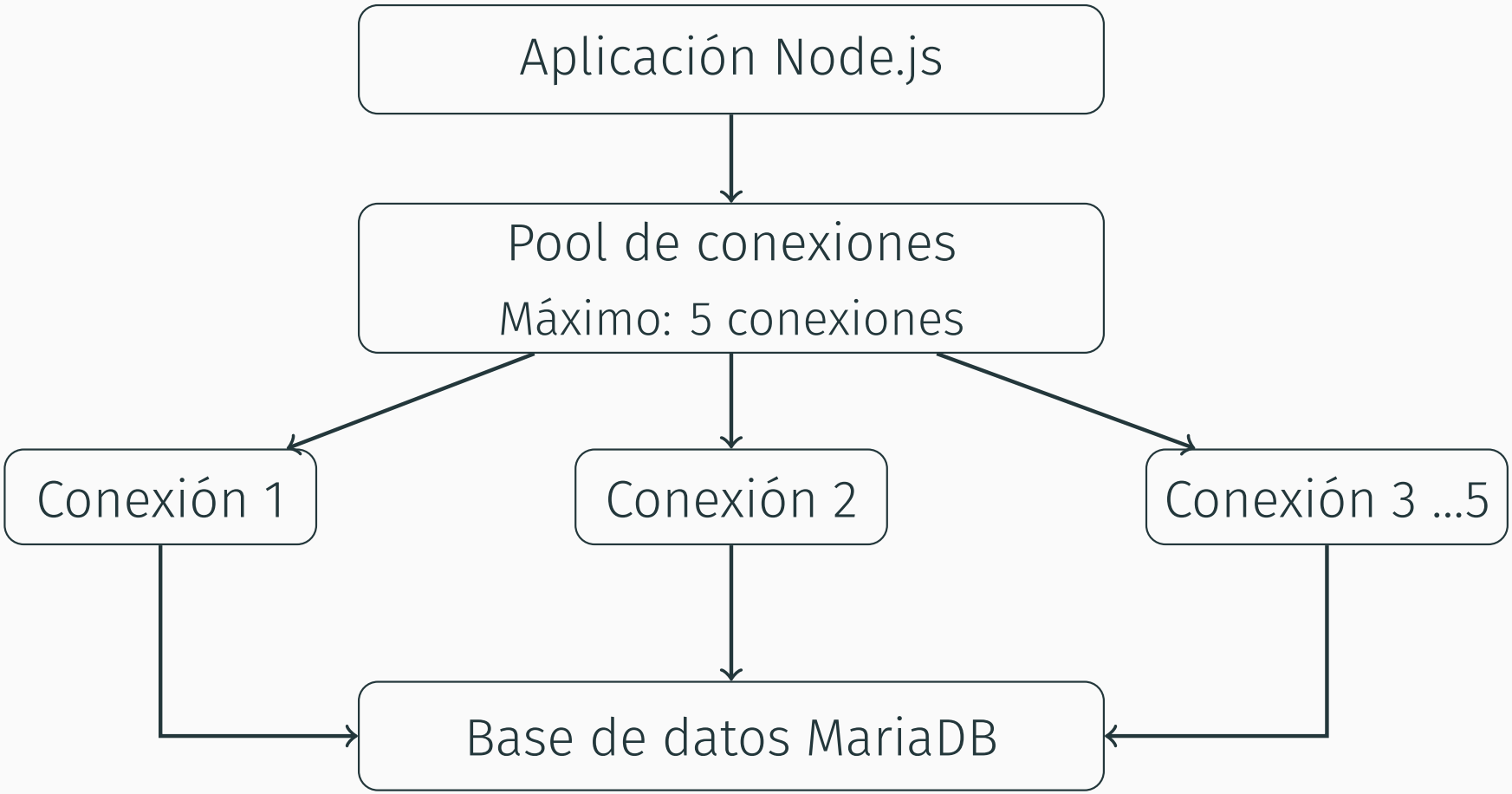
Un *pool* es un conjunto de conexiones a la base de datos que la aplicación mantiene disponibles para reutilizarlas.

Problema
Crear una conexión nueva para cada consulta genera sobrecarga de red y consumo de recursos.

Solución
Utilizar conexiones reutilizables y establecer un límite máximo de conexiones simultáneas.

- **Reutilización:** evita crear una conexión para cada consulta.
- **Límite:** controla las conexiones simultáneas.
- **Espera:** las solicitudes esperan si no hay conexiones disponibles.

Funcionamiento del pool



Preparación del proyecto

Creamos un proyecto de Node.js e instalamos el conector de MariaDB y el paquete para manejar variables de entorno.

Comandos en la terminal

```
mkdir pool-mariadb
cd pool-mariadb

npm init -y
npm pkg set type="module"
npm install mariadb dotenv
```

Archivos del proyecto

- `.env`: credenciales de conexión.
- `database.js`: configuración del pool.
- `index.js`: consultas a la base de datos.

Crear la base de datos

Ejecutamos las siguientes sentencias SQL en MariaDB.

Base de datos y tabla

```
CREATE DATABASE escuela;
USE escuela;

CREATE TABLE usuarios (
id INT AUTO_INCREMENT PRIMARY KEY,
nombre VARCHAR(100) NOT NULL,
email VARCHAR(150) NOT NULL UNIQUE
);

INSERT INTO usuarios (nombre, email) VALUES
('Ana Lopez', 'ana@ejemplo.com'),
('Pedro Garcia', 'pedro@ejemplo.com'),
('Maria Torres', 'maria@ejemplo.com');
```

Configurar las credenciales

Creamos el archivo `.env` en la raíz del proyecto.

Archivo .env

```
DB_HOST=localhost
DB_PORT=3306
DB_USER=app_usuario
DB_PASSWORD=ClaveDePrueba123!
DB_NAME=escuela
```

Importante
El usuario de MariaDB debe existir y contar con los permisos necesarios sobre la base de datos. No se recomienda utilizar la cuenta `root` desde la aplicación.

Configuración del pool en Node.js

Archivo: `database.js`

Código

```
import mariadb from 'mariadb';
import dotenv from 'dotenv';

dotenv.config();

const pool = mariadb.createPool({
  host: process.env.DB_HOST,
  port: Number(process.env.DB_PORT),
  user: process.env.DB_USER,
  password: process.env.DB_PASSWORD,
  database: process.env.DB_NAME,
  connectionLimit: 5,
  acquireTimeout: 10000
});

export default pool;
```

Parámetros principales del pool

Parámetro	Descripción
`host`	Dirección del servidor MariaDB.
`port`	Puerto de conexión, normalmente 3306.
`database`	Base de datos que utilizará la aplicación.
`connectionLimit`	Máximo de conexiones simultáneas del pool.
`acquireTimeout`	Tiempo máximo de espera para adquirir una conexión, en milisegundos.

Observación

El pool administra las conexiones disponibles. No es necesario abrir manualmente una conexión para cada consulta cuando se utiliza `pool.query()`.

Consultar los registros

Archivo: `index.js`

Código

```
import pool from './database.js';

async function obtenerUsuarios() {
  try {
    const usuarios = await pool.query(
    'SELECT id, nombre, email FROM usuarios');
    console.table(usuarios);
  } catch (error) {
    console.error('Error:', error.message);
  } finally {
    await pool.end();
  }
}
obtenerUsuarios();
```

Ejecutar el programa

Desde la terminal, dentro del directorio del proyecto:

Comando

```
node index.js
```

Resultado esperado

La terminal mostrará una tabla con los identificadores, nombres y correos electrónicos de los usuarios registrados en MariaDB.

- Si la consulta funciona, se muestran los registros.
- Si falla la conexión, se informa el error.
- Al finalizar, `pool.end()` cierra el pool.

Sentencias parametrizadas

Podemos consultar un usuario mediante su identificador sin concatenar directamente el valor en la consulta SQL.

Consulta por identificador

```
const usuarios = await pool.query(
'SELECT id, nombre, email FROM usuarios WHERE id = ?',[2]);
console.table(usuarios);
```

- El símbolo ? representa un parámetro.
- El arreglo proporciona los valores correspondientes.
- El conector separa los valores de la estructura SQL.

Buena práctica

Utilizar consultas parametrizadas ayuda a prevenir inyecciones SQL.

Prueba de consultas concurrentes

Ejecutamos diez consultas y dejamos que el pool administre las conexiones disponibles.

Código

```
const tareas = Array.from(
{ length: 10 },
(_, i) => pool.query('SELECT ? AS consulta',[i + 1]));
const resultados = await Promise.all(tareas);
console.log('Consultas completadas:',resultados.length);
```

¿Qué debemos observar?

Se solicitan diez consultas, pero el pool tiene un límite de cinco conexiones simultáneas. Las solicitudes que no obtengan una conexión disponible deberán esperar.

Actividad de comprobación

Instrucciones

Modifica el programa para realizar las siguientes actividades:

1. Cambia el límite del pool a tres conexiones.
2. Consulta un usuario mediante su identificador.
3. Prueba con un identificador que no exista.
4. Ejecuta cinco consultas concurrentes.
5. Explica qué problema resuelve el pool.

Preguntas de reflexión

- ¿Qué diferencia existe entre una conexión y un pool?
- ¿Por qué no conviene abrir una conexión por consulta?
- ¿Qué función cumple el parámetro `connectionLimit`?

Mapeo objeto-relacional (ORM)

Estrategias de mapeo de entidades y relaciones

Concepto de ORM

Mecanismo que permite abstraer las tablas de una base de datos relacional en clases u objetos del lenguaje de programación.

- **Mapeo Entidad-Clase**: Cada tabla se mapea a una clase; cada fila se mapea a una instancia de dicho objeto.
- **Mapeo de Relaciones**:
  - **1:1 (Uno a Uno)**: Clave foránea única en una de las tablas.
  - **1:N (Uno a Muchos)**: Clave foránea referenciando a la entidad principal.
  - **N:M (Muchos a Muchos)**: Requiere una tabla intermedia de unión (*pivot table*).
- **Tipado e Integridad**: Conversión automática de tipos de datos SQL a tipos nativos del lenguaje.

Estrategias de carga: Lazy Loading vs. Eager Loading

Forma en la que el ORM recupera los datos de entidades relacionadas:

Carga Perezosa (Lazy Loading)

- Las relaciones se cargan bajo demanda cuando se accede a la propiedad.
- Retrasa consultas innecesarias.
- *Riesgo*: Puede desencadenar múltiples peticiones ocultas a la base de datos.

Carga Ansiosa (Eager Loading)

- Trae la entidad principal y sus relaciones en una sola consulta (`JOIN` o subconsulta).
- Mayor costo inicial de transferencia de datos.
- *Ventaja:* Predictibilidad en el número de consultas.

El problema N+1 de rendimiento

Definición del problema
Ocurre al iterar sobre N registros ejecutando una consulta adicional por cada uno para recuperar sus relaciones, totalizando $N + 1$ consultas a la base de datos.

- **Ejemplo**: Consulta de 100 usuarios (1 consulta) y luego acceso a los posts de cada usuario en un bucle (100 consultas adicionales) $=$ 101 consultas.
- **Impacto**: Degradación severa del tiempo de respuesta y saturación del pool de conexiones.
- **Solución**: Utilizar explicitamente **Eager Loading** (ej. `include` en Prisma o `joinedload` en SQLAlchemy) o batching de IDs (`WHERE id IN (...)`).

Patrón Repository sobre un ORM

Aislamiento de la infraestructura
Aunque un ORM abstrae SQL, desacoplar el ORM de la capa de servicio mediante el patrón Repository evita la dependencia directa con una librería específica.

- **Independencia**: Permite migrar de ORM (o modificar su versión) sin alterar las reglas de negocio.
- **Facilidad de Testing**: Facilita el reemplazo del repositorio real por implementaciones en memoria (*Mocks*) en pruebas unitarias.
- **Control**: Define métodos con nombres expresivos de dominio en lugar de exponer métodos genéricos del ORM.

¿Cuándo recurrir a SQL nativo?

A pesar de los beneficios del ORM, existen escenarios donde usar SQL directo es necesario:

- **Consultas complejas de analítica**: Uso de funciones de ventana (*Window Functions*), CTEs complejas o agregaciones masivas.
- **Optimización estricta de rendimiento**: Evitar el costo de hidratación de miles de objetos en memoria por parte del ORM.
- **Operaciones en lote (Bulk Operations)**: Inserciones o actualizaciones masivas donde el ORM procesa entidad por entidad.
- **Funcionalidades específicas del motor**: Funciones geoespaciales avanzadas (PostGIS), búsquedas de texto completo (*Full-Text Search*) o tipos JSON nativos.

Práctica: Inicialización de ORM en proyectos

Node.js (Prisma ORM)

```
# Crear directorio y entrar
mkdir orm-practica && cd orm-practica
npm init -y
npm pkg set type="module"

# Instalar Prisma
npm install @prisma/client
npm install -D prisma

# Inicializar configuración
npx prisma init
```

Python (SQLAlchemy 2.0)

```
# Crear directorio y entrar
mkdir orm-practica && cd orm-practica
python -m venv venv
source venv/bin/activate

# Instalar SQLAlchemy y driver
pip install sqlalchemy \
            psycopg2-binary \
            python-dotenv
```

Práctica: Eager Loading y prevención de N+1 en Node.js

```
import { PrismaClient } from '@prisma/client';
const prisma = new PrismaClient();

export async function obtenerUsuariosConPosts() {
  // Eager Loading con 'include' previene el problema N+1
  const usuarios = await prisma.usuario.findMany({
    include: {
      posts: true // Trae la relación en una sola consulta optimizada
    }
  });
  return usuarios;
}

// Ejemplo de consulta raw (SQL Nativo)
export async function reporteComplejo() {
  return await prisma.$queryRaw`
    SELECT u.nombre, COUNT(p.id) as total_posts
    FROM "Usuario" u JOIN "Post" p ON u.id = p.usuario_id
    GROUP BY u.id HAVING COUNT(p.id) > 5;
  `;
}
```

Práctica: Patrón Repository con Prisma en Node.js

```
export class UsuarioRepositoryPrisma {
  constructor(prismaClient) {
    this.prisma = prismaClient;
  }

  async findByIdWithPosts(id) {
    return await this.prisma.usuario.findUnique({
      where: { id: Number(id) },
      include: { posts: true }
    });
  }

  async create(data) {
    return await this.prisma.usuario.create({ data });
  }
}
```

Práctica: Eager Loading y N+1 en Python (SQLAlchemy)

```
from sqlalchemy import create_engine, select, text
from sqlalchemy.orm import Session, joinedload
from models import Usuario, Post

engine = create_engine("postgresql://usuario:pass@localhost:5432/db")

def obtener_usuarios_con_posts():
    with Session(engine) as session:
        # joinedload realiza un JOIN para prevenir el problema N+1
        stmt = select(Usuario).options(joinedload(Usuario.posts))
        usuarios = session.scalars(stmt).unique().all()
        return usuarios

def reporte_sql_nativo():
    with Session(engine) as session:
        # Consulta SQL nativa para agregaciones avanzadas
        query = text("SELECT usuario_id, COUNT(*) FROM posts GROUP BY usuario_id;")
        return session.execute(query).fetchall()
```

Práctica: Patrón Repository con SQLAlchemy en Python

```
class UsuarioRepositorySQLAlchemy:
    def __init__(self, session: Session):
        self.session = session

    def get_by_id_with_posts(self, user_id: int):
        stmt = (
            select(Usuario)
            .where(Usuario.id == user_id)
            .options(joinedload(Usuario.posts))
        )
        return self.session.scalars(stmt).first()

    def save(self, usuario: Usuario):
        self.session.add(usuario)
        self.session.commit()
        return usuario
```

Migraciones de base de datos

Control de versiones del esquema de datos

Concepto de Migración

Las migraciones son archivos de código o SQL con marcas de tiempo que documentan la evolución incremental y reversible del esquema de una base de datos.

- **Evolución sincronizada**: Garantizan que la estructura de la base de datos avance a la par del código fuente de la aplicación en todos los entornos.
- **Reproducibilidad**: Permiten recrear una base de datos desde cero en cualquier estado histórico ejecutando la secuencia de migraciones.
- **Estructura de una migración**:
  - **UP**: Define los cambios a aplicar (crear tabla, agregar columna, añadir índice).
  - **DOWN**: Define la lógica inversa para revertir los cambios (*rollback*).

Buenas prácticas en el control de versiones del esquema

Estrategias para mantener la integridad de los datos en entornos compartidos y de producción:

- **Inmutabilidad**: Una vez aplicada y compartida en el repositorio (`main`), una migración NUNCA debe modificarse ni eliminarse.
- **Cambios destructivos con precaución**: Evitar eliminar columnas o tablas en uso activo; emplear un enfoque en dos fases (deprecar $\rightarrow$ eliminar).
- **Tabla de estado (`_migrations`)**: La base de datos mantiene una tabla interna que registra qué archivos de migración ya han sido ejecutados.
- **Automatización en CI/CD**: Ejecución automática de las migraciones pendientes durante el proceso de despliegue antes de iniciar la nueva versión de la app.

Herramientas de migración populares

Comparativa de enfoques y herramientas según el ecosistema tecnológico:

Basadas en Declaración/Schema

- Se define el estado deseado en un esquema central; la herramienta calcula las diferencias SQL.
- **Ejemplos**: Prisma Migrate (Node.js), Django Migrations (Python).

Basadas en Scripts / SQL

- Archivos SQL o código secuencial con funciones explícitas `up()` y `down()`.
- **Ejemplos**: Alembic (Python/SQLAlchemy), Knex.js / TypeORM (Node.js), Flyway, Liquibase.

Práctica: Inicialización de Herramientas de Migración

Node.js (Prisma Migrate)

```
# Inicializar entorno
mkdir migraciones-demo && cd migraciones-demo
npm init -y
npm pkg set type="module"
npm install @prisma/client
npm install -D prisma

# Generar archivo prisma/schema.prisma
npx prisma init
```

Python (Alembic + SQLAlchemy)

```
# Inicializar entorno
mkdir migraciones-demo && cd migraciones-demo
python -m venv venv
source venv/bin/activate
pip install alembic sqlalchemy psycopg2-binary

# Inicializar repositorio de Alembic
alembic init alembic
```

Práctica: Flujo de Migraciones con Prisma (Node.js)

```
// prisma/schema.prisma
datasource db {
  provider = "postgresql"
  url      = env("DATABASE_URL")
}

model Usuario {
  id        Int      @id @default(autoincrement())
  email     str      @unique
  nombre    String
  creadoEn  DateTime @default(now())
}

# Comandos de CLI en terminal:
# 1. Crear y aplicar migración en desarrollo
# npx prisma migrate dev --name init_usuarios

# 2. Aplicar migraciones pendientes en producción
# npx prisma migrate deploy
```

Práctica: Definición manual de migración en Node.js (Knex.js)

```
// migrations/20260926_crear_tabla_usuarios.js

export async function up(knex) {
  return knex.schema.createTable('usuarios', (table) => {
    table.increments('id').primary();
    table.string('email').notNullable().unique();
    table.string('nombre').notNullable();
    table.timestamps(true, true); // creado_en y actualizado_en
  });
}

export async function down(knex) {
  return knex.schema.dropTableIfExists('usuarios');
}

// Ejecución CLI: npx knex migrate:latest / npx knex migrate:rollback
```

Práctica: Configuración y Auto-generación en Python (Alembic)

```
# alembic/env.py
from logging.config import fileConfig
from sqlalchemy import engine_from_config, pool
from alembic import context
import os

# Importar la Base de los modelos de SQLAlchemy
from mymodels import Base
target_metadata = Base.metadata

# Configurar URL de BD desde variable de entorno
config = context.config
config.set_main_option('sqlalchemy.url', os.getenv('DATABASE_URL'))
```

Práctica: Ejecución de Migraciones con Alembic (Python)

```
# alembic/versions/1a2b3c_crear_tabla_usuarios.py (Auto-generada o manual)
from alembic import op
import sqlalchemy as sa

def upgrade():
    op.create_table(
        'usuarios',
        sa.Column('id', sa.Integer(), nullable=False, primary_key=True),
        sa.Column('email', sa.String(length=255), nullable=False, unique=True),
        sa.Column('nombre', sa.String(length=100), nullable=False)
    )

def downgrade():
    op.drop_table('usuarios')

# Comandos CLI:
# alembic revision --autogenerate -m "crear tabla usuarios"
# alembic upgrade head
```

Caché y almacenes NoSQL

Almacenes clave-valor para caché y sesiones

¿Por qué memoria primaria (In-Memory)?

Los almacenes de datos en memoria (como Redis o Memcached) ofrecen lecturas y escrituras con latencias de sub-milisegundo al omitir el I/O de disco.

- **Caché de datos (Query Caching)**: Almacena resultados de consultas costosas a la base de datos relacional para servir solicitudes subsecuentes de forma instantánea.
- **Gestión de Sesiones**: Mantiene el estado de autenticación centralizado, permitiendo un escalado horizontal transparente del back-end sin dependencia de instancias específicas.
- **Estrategias de Expiración (TTL)**: Configuración de *Time-To-Live* e invalidador de caché para prevenir la presencia de datos obsoletos (*stale data*).
- **Estructuras de datos avanzadas**: Manejo nativo de listas, conjuntos, *hashes* y rangos ordenados en Redis.

Bases de datos documentales: Criterios de uso

Almacenamiento orientado a documentos (JSON/BSON)
Permiten almacenar datos semiestructurados donde cada registro (documento) encapsula su propia información sin exigir un esquema rígido a nivel de motor.

- **Cuándo utilizarlas**:
  - Esquemas altamente dinámicos o cambiantes en el tiempo.
  - Estructuras jerárquicas o anidadas que se leen en conjunto con frecuencia (ej. catálogo de productos con atributos variables, logs, carritos).
  - Requerimientos de alta velocidad en escrituras masivas.
- **Cuándo NO utilizarlas**:
  - Sistemas con relaciones complejas de tipo M:N o requerimientos estrictos de integridad referencial.
  - Transacciones financieras complejas entre múltiples entidades desacopladas.

Arquitecturas de datos híbridas (Polyglot Persistence)

Persistencia Políglota

Consiste en utilizar múltiples tecnologías de bases de datos dentro de una misma arquitectura back-end, seleccionando el motor óptimo para cada tipo de problema.

- **PostgreSQL / MySQL**: Transacciones financieras, datos de usuarios y relaciones maestras (Garantía ACID).
- **Redis**: Caché de aplicación, límites de tasa (*rate limiting*), colas rápidas y sesiones.
- **MongoDB / DocumentDB**: Logs de auditoría, eventos de analítica y perfiles de producto dinámicos.
- **Desafío principal**: Mantener la consistencia eventual entre los diferentes almacenes y gestionar la complejidad operativa.

Práctica: Inicialización de clientes Redis y MongoDB

Node.js (**redis** y **mongodb**)

```
# Crear directorio y entrar
mkdir nosql-practica && cd nosql-practica
npm init -y
npm pkg set type="module"

# Instalar drivers
npm install redis mongodb dotenv
```

Python (**redis** y **pymongo**)

```
# Crear directorio y entrar
mkdir nosql-practica && cd nosql-practica
python -m venv venv
source venv/bin/activate

# Instalar librerías
pip install redis pymongo \
            python-dotenv
```

Práctica: Caché Clave-Valor con Redis en Node.js

```
import { createClient } from 'redis';

const redisClient = createClient({ url: process.env.REDIS_URL });
await redisClient.connect();

export async function obtenerPerfilUsuario(userId, dbQueryCallback) {
  const cacheKey = `user:${userId}`;

  // 1. Intentar obtener del caché de Redis
  const cachedData = await redisClient.get(cacheKey);
  if (cachedData) {
    return JSON.parse(cachedData); // Cache Hit
  }

  // 2. Cache Miss: Consultar la base de datos principal
  const user = await dbQueryCallback(userId);

  // 3. Guardar en Redis con TTL de 3600 segundos (1 hora)
  await redisClient.setEx(cacheKey, 3600, JSON.stringify(user));
  return user;
}
```

Práctica: Operaciones Documentales con MongoDB en Node.js

```
import { MongoClient } from 'mongodb';

const mongoClient = new MongoClient(process.env.MONGO_URI);
await mongoClient.connect();
const db = mongoClient.db('tienda');
const productosCol = db.collection('productos');

export async function crearProductoDinamico(productoData) {
  // Inserta documentos JSON con atributos arbitrarios
  const resultado = await productosCol.insertOne({
    ...productoData,
    creadoEn: new Date()
  });
  return resultado.insertedId;
}

export async function buscarPorCategoria(categoria) {
  return await productosCol.find({ categoria }).toArray();
}
```

Práctica: Caché Clave-Valor con Redis en Python

```
import json
import redis
import os

redis_client = redis.Redis.from_url(os.getenv("REDIS_URL", "redis://localhost:6379"))

def obtener_perfil_usuario(user_id: int, db_query_func):
    cache_key = f"user:{user_id}"

    # 1. Intentar lectura de caché
    cached_data = redis_client.get(cache_key)
    if cached_data:
        return json.loads(cached_data)  # Cache Hit

    # 2. Cache Miss: Consultar BD relacional
    user_data = db_query_func(user_id)

    # 3. Almacenar en Redis con expiración (TTL = 3600s)
    redis_client.setex(cache_key, 3600, json.dumps(user_data))
    return user_data
```

Práctica: Operaciones Documentales con PyMongo en Python

```
from pymongo import MongoClient
import os

mongo_client = MongoClient(os.getenv("MONGO_URI", "mongodb://localhost:27017"))
db = mongo_client["tienda"]
productos_col = db["productos"]

def crear_producto_dinamico(producto_data: dict):
    # Inserción de documento semiestructurado
    result = productos_col.insert_one(producto_data)
    return str(result.inserted_id)

def buscar_por_atributos_dinamicos(filtros: dict):
    # Consulta flexible basada en los campos del documento JSON
    return list(productos_col.find(filtros, {"_id": 0}))
```